UNIVERSIDAD
DE SANTIAGO
DE CHILE

Informe 1 Paradigmas de Programación
Paradigma Funcional Scheme

Nombre	RUT
Raúl Alberto González Véliz	20.994.246-1

Profesor: Edmundo Leiva
Curso: 13204-0-A-1
Carrera: Ingeniería en Ejecución en Computación e Informática
Fecha de la entrega: 31/10/2024

UNIVERSIDAD
DE SANTIAGO
DE CHILE

Tabla de Contenidos

Introducción 3
Descripción del Problema 4
Descripción del Paradigma 4
Análisis del Problema 4
Diseño de la Solución 5
Aspectos de Implementación 6
Instrucciones de Uso 6
Conclusiones 7
Referencias 8
Anexos 9

Introducción

En el presente informe se abordará un proyecto, el cual consiste en generar un juego completamente funcional de connect 4, esto utilizando el famoso lenguaje de programación enfocado en el paradigma funcional "scheme". La primera pregunta que surge es: ¿Qué es el connect 4? El connect 4 constituye un juego bastante simple. En el cual requerimos:

- Dos jugadores.
- Un tablero de 7 columnas por 6 filas.
- Un mínimo de 4 fichas por cada color (generalmente se juega con fichas rojas y amarillas, pero este proyecto permite elegir el color que sea de la preferencia del usuario siempre y cuando los colores sean distintos) y un máximo de 21, pues hay 42 espacios en el tablero.

Y las condiciones para ganar son las siguientes:

- Que se encuentren 4 fichas del mismo color seguidas en una columna (victoria vertical).
- Que se encuentren 4 fichas del mismo color seguidas en una fila (victoria horizontal).
- Que se encuentren 4 fichas del mismo color seguidas en una diagonal del tablero (victoria diagonal).
- Para que haya un empate ambos jugadores se deben quedar sin fichas sin haber logrado conectar 4 fichas en línea.

Descripción del Problema

Se debe implementar un juego de connect 4 sólo utilizando el paradigma funcional en el lenguaje de programación scheme. A esta implementación se le debe poder agregar fichas con colores (procurando cambiar el turno de los jugadores cuando la jugada sea realizada) y verificar si nos encontramos en una situación de victoria de alguno de los 2 jugadores o si el tablero está empatado (ya sea por falta de fichas o tablero lleno). También debe tener un historial que registre la columna en que se jugó la ficha y el color de dicha ficha.

Descripción del Paradigma

El paradigma funcional, tal como su nombre lo indica, consiste en programar única y exclusivamente utilizando funciones para lograr el fin requerido, teniendo como característica principal la recursividad (ya sea natural o de cola), dejando de lado todo lo que corresponda a ciclos como el while o el for in, así como también se prescinde de la asignación de variables, característica insigne del paradigma procedural imperativo. El paradigma funcional es de tipo declarativo, pues se especifica al programa lo que tiene que hacer.

Análisis del Problema

Para este problema se indica la creación de 4 TDA´s básicos requeridos por el enunciado:

- TDA player.
- TDA piece.
- TDA board.
- TDA game.

Para lograr implementar los anteriores TDA's mencionados y obtener las salidas esperadas según los requerimientos funcionales (RF) y las entradas de estos será necesario implementar otros TDA's para lograr el fin requerido.

Considerando que cada TDA debe tener sus propias funciones, el uso del *car, cdr, null?* y demás funciones nativas de scheme deben ser utilizadas a través de funciones propias redefinidas que corresponderá a los getters, modifiers y setters de cada TDA.

Algunos TDA's tendrán como base otros TDA's para poder tener una representación correcta en la implementación del proyecto y lograr una mayor facilidad a la hora de realizar los requisitos funcionales.

Diseño de la Solución

En base a lo analizado en el punto anterior, el diseño de la solución será con la siguiente estructura:

TDA Game: será construido a partir de los demás TDA's, este está constituido por los dos jugadores (ambos TDA player), el tablero del juego en curso (TDA board), un entero que indica el turno en que se encuentra el juego (el turno corresponderá al ID del jugador cuyo turno esté en curso, si un jugador intenta jugar fuera de su turno su movimiento no será válido) y un historial el cuál se irá actualizando a medida que se vayan jugando las piezas en el tablero.

TDA board: corresponde a una lista de 7 columnas (otro TDA) y es donde son insertadas las fichas de connect 4 como en forma de TDA position. A su vez el TDA board "muta" en TDA row-board, que corresponderá al tablero invertido, esto quiere decir que en vez de columnas, ahora será una lista de 6 filas (TDA row), esto será útil para verificar la victoria horizontal.

TDA player: este está compuesto por 7 elementos, los cuales son: ID del jugador, el cual indicará que turno le corresponde jugar a cada jugador, el nombre del jugador, su color, la cantidad de victorias, derrotas, empates y fichas restantes del jugador (ninguno de estos corresponde un TDA en sí.)

TDA column: indica una columna del tablero, este estará compuesto por los TDA position y sirven de base para construir el TDA board.

TDA row: bastante similar al TDA anterior, sólo que está construido únicamente por los TDA piece y son la base para construir el TDA row-board.

TDA position: este comprende un par entre número de fila en que se encuentra la ficha y el TDA piece, además de ser la base del TDA column.

TDA piece: básicamente es un par de color de ficha e ID del jugador al cual corresponde el color.

Aspectos de Implementación

El proyecto enfocado hacia construir el connect 4 y dejarlo funcional está compuesto por 8 archivos, cada uno implementando los TDA's mencionados en el Diseño de la Solución. Estos son ejecutados en los 3 scripts de prueba (2 propios y el otro proporcionado por el enunciado del proyecto) mostrando la ejecución de los requerimientos funcionales solicitados para el proyecto.

A su vez el proyecto fue realizado en el compilador DrRacket, para ser más exactos en su versión 8.14 a través del lenguaje de programación scheme sin utilizar alguna biblioteca externa, pues era requisito utilizar sólo las funciones nativas de scheme.

Instrucciones de Uso

1. Es sumamente importante y obligatorio que los archivos TDA y los scripts de prueba estén en la misma carpeta para que los *require/provide* funcionen de manera correcta.
2. Considerar que al momento de insertar las fichas en la función **game-player-set-move** el número de columna indicado debe ser entre 1 y 7.
3. Al momento de inicializar el game se debe indicar de manera correcta el ID de los jugadores (tiene que ser 1 o 2), que los jugadores no tengan el mismo color, que su cantidad de fichas sea menor o igual a 21, y el turno debe corresponder con el ID de alguno de los jugadores (1 o 2). De no cumplirse alguna de estas condiciones la función game considerará inválido el juego ingresado indicando el error con el display.
4. En el primer juego se podrá inicializar con una cantidad de fichas arbitraria menor a 22, sin embargo, las siguientes partidas dentro de ese game serán inicializadas con 21 fichas para cada jugador.

Conclusiones

Los objetivos propuestos fueron alcanzados satisfactoriamente, pues se logró implementar los requerimientos funcionales con éxito y su vez estos lograron los resultados esperados en el script de pruebas proporcionado en el enunciado del proyecto, solo cambiando un pequeño detalle, pues el script de pruebas coloca piezas tomando en cuenta que las columnas del tablero van del 0 al 6, sin embargo la implementación realizada cuenta estás desde el 1 hasta el 7. Para solventar este pequeño inconveniente se modificó el script de pruebas sumándole 1 a cada columna en la cual se colocaba cada ficha (si una instrucción la ficha se colocaba originalmente en la columna 3, ahora esa misma instrucción indica que la ficha sea colocada en la columna 4), y esto último permite respetar la idea de tablero que se tenía pensada en el script de pruebas.

UNIVERSIDAD
DE SANTIAGO
DE CHILE

Referencias

Hasbro. (s.f.). Connect Four [Instrucciones del juego]. https://www.hasbro.com/common/instruct/ConnectFour.PDF

Martinez, Gonzalo. “Proyecto semestral de laboratorio.” Septiembre 2024, https://docs.google.com/document/d/1VzRjW_TrsOZNG1LNMuXo85IEhRc3aCQat16cipalMiU/edit?tab=t.0.

Martinez, Gonzalo. “Proyecto semestral de laboratorio.” Septiembre 2024, https://docs.google.com/document/d/1VzRjW_TrsOZNG1LNMuXo85IEhRc3aCQat16cipalMiU/edit?tab=t.0.

Anexos

```
; 1. Creación de jugadores (7 fichas cada uno para un juego de corta duración)
(define p1 (player 1 "GokU" "NARANJA" 0 0 0 7))
(define p2 (player 2 "VegeTA" "azUl" 0 0 0 7))

; 2. Creación de piezas
(define barca-piece (piece "NARANJA"))
(define madrid-piece (piece "azUl"))
```

```
; 5. Realizando movimientos para crear un empate por falta de fichas

(define g1 (game-player-set-move g0 (game-get-current-player g0) 1))  ; Vegeta coloca en columna 1
(define g2 (game-player-set-move g1 (game-get-current-player g1) 2))  ; Goku coloca en columna 2
(define g3 (game-player-set-move g2 (game-get-current-player g2) 1))  ; Vegeta coloca en columna 1
(define g4 (game-player-set-move g3 (game-get-current-player g3) 2))  ; Goku coloca en columna 2
(define g5 (game-player-set-move g4 (game-get-current-player g4) 2))  ; Vegeta coloca en columna 2
(define g6 (game-player-set-move g5 (game-get-current-player g5) 4))  ; Goku coloca en columna 1
(define g7 (game-player-set-move g6 (game-get-current-player g6) 5))  ; Vegeta coloca en columna 2
(define g8 (game-player-set-move g7 (game-get-current-player g7) 6))  ; Goku coloca en columna 1
(define g9 (game-player-set-move g8 (game-get-current-player g8) 2))  ; Vegeta coloca en columna 1
(define g10 (game-player-set-move g9 (game-get-current-player g9) 7)) ; Goku coloca en columna 2
(define g11 (game-player-set-move g10 (game-get-current-player g10) 3)) ; Vegeta coloca en columna 1
(define g12 (game-player-set-move g11 (game-get-current-player g11) 6))  ; Goku coloca en columna 1
(define g13 (game-player-set-move g12 (game-get-current-player g12) 4))  ; Vegeta coloca en columna 1
(define g14 (game-player-set-move g13 (game-get-current-player g13) 3)) ; Goku coloca en columna 2 (no se pueden jugar más fichas)
```

```
¿Se puede jugar en el tablero vacío? #t
¿Se puede jugar después de 14 movimientos? #t
Jugador actual después de 14 movimientos: (2 "vegeta" "azul" 0 0 0 0)
Verificación de victoria vertical: 0
Verificación de victoria horizontal: 0
Verificación de victoria diagonal: 0
Verificación de ganador: 0
¿Es empate? #t
Historial de movimientos: (("Fin de juego. Generando siguiente tablero vacío...")
 (3 . "naranja")
 (4 . "azul")
 (6 . "naranja")
 (3 . "azul")
 (7 . "naranja")
 (2 . "azul")
 (6 . "naranja")
 (5 . "azul")
 (4 . "naranja")
 (2 . "azul")
 (2 . "naranja")
 (1 . "azul")
 (2 . "naranja")
 (1 . "azul")
 ("Tablero vacío"))
Estado final del tablero: (((2 "azul" . 2) (1 "azul" . 2))
 ((4 "azul" . 2) (3 "azul" . 2) (2 "naranja" . 1) (1 "naranja" . 1))
 ((2 "naranja" . 1) (1 "azul" . 2))
 ((2 "azul" . 2) (1 "naranja" . 1))
 ((1 "azul" . 2))
 ((2 "naranja" . 1) (1 "naranja" . 1))
 ((1 "naranja" . 1)))
>
```

```
> (define jug-1(player 1 "GokU" "NARANJA" 0 0 0 23))
> (define jug-2(player 2 "VegeTA" "azUl" 0 0 0 23))
> (define juego(game jug-1 jug-2 (board) 2))
Juego inválido (Uno o ambos jugadores poseen una cantidad de fichas inválida). Inténtelo de nuevo.
```

```
> (define jug-1(player 1 "GokU" "NARANJA" 0 0 0 21))
> (define jug-2(player 2 "VegeTA" "naranja" 0 0 0 21))
> (define juego(game jug-1 jug-2 (board) 2))
Juego inválido (ambos jugadores tienen el mismo color de ficha). Inténtelo de nuevo.
```

```
> (define jug-1(player 1 "GokU" "NARANJA" 0 0 0 21))
(define jug-2(player 2 "VegeTA" "azUl" 0 0 0 21))
(define juego(game jug-1 jug-2 (board) 3))
Juego inválido (el turno no es válido para ninguno de los jugadores). Inténtelo de nuevo.
```